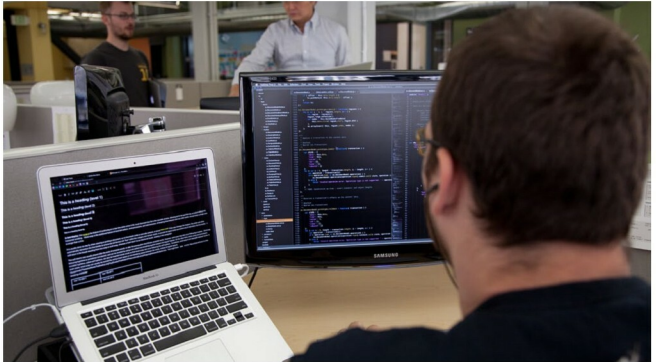


What is the Haskell I/O?

This article is a must read for anyone interested in getting a good insight into the input/output (IO) functionality of Haskell.



Input/output (IO) can cause side-effects and hence is implemented as a Monad. The IO Monad takes some input, does some computation and returns a value. The IO action is performed inside a main function. Consider a simple 'Hello world' example:

```
main = putStrLn "Hello, World!"
```

Executing the above code in GHCi produces the following output:

```
$ ghci hello.hs
GHCi, version 7.6.3: http://www.haskell.org/ghc/? for help
Loading package ghc-prim ... linking ... done.
Loading package integer-gmp ... linking ... done.
Loading package base ... linking ... done.
[1 of 1] Compiling Main ( foo.hs, interpreted )
Ok, modules loaded: Main.
ghci> main
Hello, World!
```

The type signatures of *main* and *putStrLn* are:

```
main :: IO ()
```

```
putStrLn :: String -> IO ()
```

putStrLn takes a string as input and prints the string to output. It doesn't return anything, and hence the return type is the empty tuple ().

The *getline* function performs an IO to return a string.

```
ghci> :t getline
getline :: IO String
```

```
ghci> name <- getline
Foo
```

```
ghci> name
"Foo"
```

The '<-' extracts the result of the IO string action, unwraps it to obtain the string value, and 'name' gets the value. So, the type of 'name' is:

```
ghci> :t name
name :: String
```

The *do* syntax is useful to chain IO together. For example: